

Fibonacci Heap

①

Recap: Heap or Priority Queue

- Binary Min-heaps/Max heaps
- Can support creating a heap, insert, finding/extracting the min/max efficiently.
- Can also support decrease key operation efficiently.
- But, Not good for merging two heaps.
 - $O(n)$ where n is total no. of elements in the two heaps
- Variations of heaps exist that can merge heaps efficiently.
 - May also improve the complexity of other operations.
 - Ex:- Binomial heaps, Fibonacci heaps.
- We study Fibonacci Heaps, an amortized data structure.

A comparison

Operation	Binary Heap (Worst case)	Fibonacci Heap, (amortized)
Make-heap	$\Theta(1)$	$\Theta(1)$
Insert	$\Theta(\log n)$ $\Theta(\log n)$	$\Theta(1)$
Minimum	$\Theta(1)$	$\Theta(1)$
Extract-Min	$\Theta(\log n)$	$\Theta(\log n)$
Merge/Union	$\Theta(n)$	$\Theta(1)$
Decrease-key	$\Theta(\log n)$	$\Theta(1)$
Delete	$\Theta(\log n)$	$\Theta(\log n)$

Fibonacci Heap

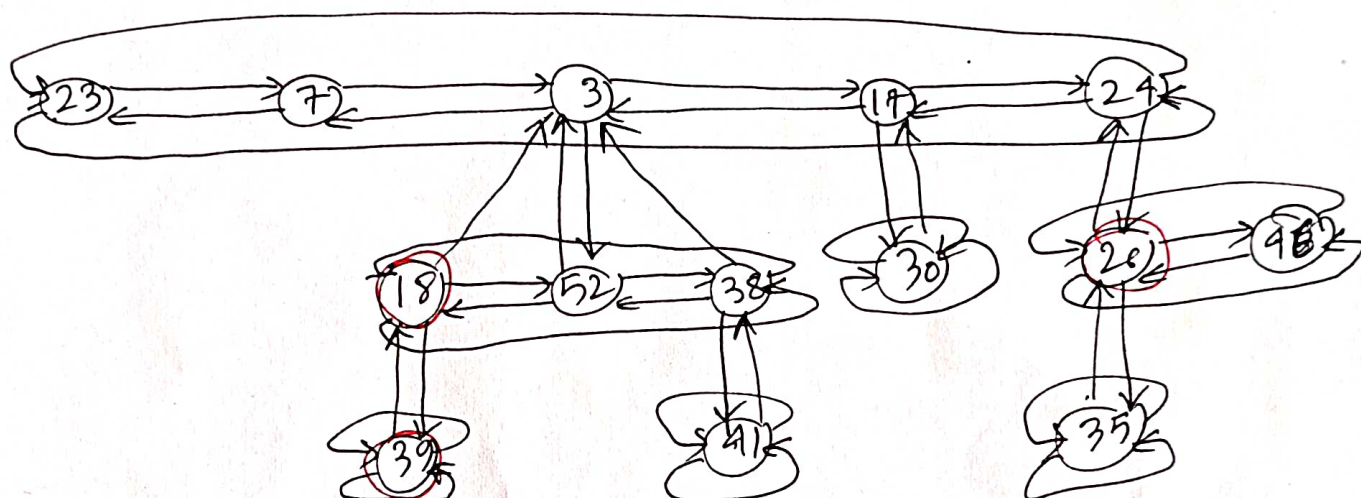
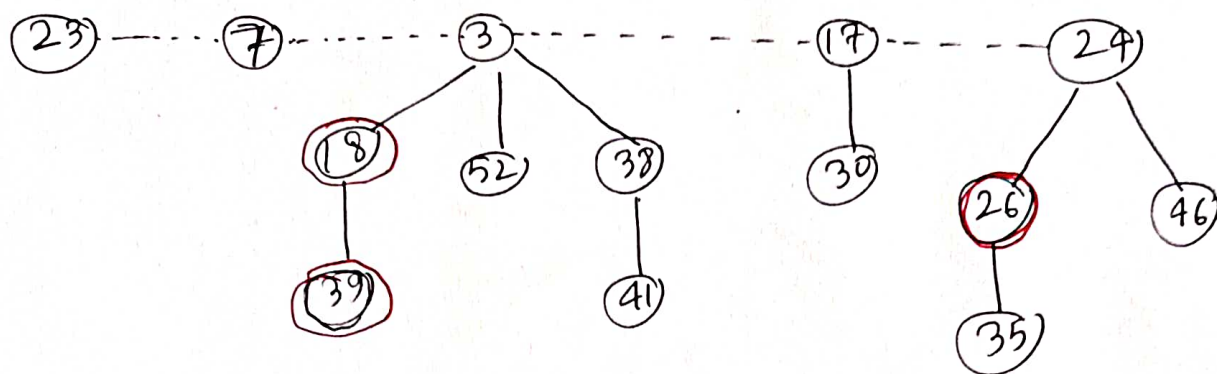
(2)

Def: * A collection of min-heap ordered trees.

- Each tree is rooted but "unordered", meaning there is no order between the child nodes of a node
- Each node x has
 - * One parent node $p[x]$ [except root nodes]
 - * One child pointer $child[x]$ which points to an arbitrary child of x .
 - * The children of x are linked together in a circular, doubly linked list.
 - Each node y has pointers $left[y]$ and $right[y]$ to its left and right node in the list
 - So, x basically stores a pointer to start in this list of its children.
- The roots of the trees are again connected with a circular, doubly linked list using their left and right pointers
- A Fibonacci heap H is defined by
 - * A pointer $min[H]$ which points to the root of a tree containing the minimum element (minimum node of the heap)
 - * A variable $n[H]$ storing the number of elements in the heap

Fibonacci Heap

(3)



Additional variables

* - Each node x also has two other fields

* $\text{degree}[x]$ - Stores the number of children of x

* $\text{mark}[x]$ - indicates whether x has lost a child since the last time x was made the child of another node.

- We will denote marked nodes by red color

- A newly created node is unmarked

- A marked node also becomes unmarked whenever it is made the child of another node

Fibonacci Heap

(4)

Amortized analysis: Potential function

- We mentioned Fibonacci heap is an amortized data structure
- We will use potential method to analyse.
- Let $t(H)$ = no. of trees in a Fibonacci heap H
- Let $m(H)$ = number of marked nodes in H
- Potential function used.

$$\Phi(H) = t(H) + 2m(H)$$

Operations on Fibonacci Heap

- * Create an empty Fibonacci heap
- * Insert an element in a Fibonacci heap
- * Merge two Fibonacci heaps (Union)
- * Extract the minimum element from a Fibonacci heap
- * Decrease the value of an element in a Fibonacci heap
- * Delete an element from a Fibonacci heap

Creating a Fibonacci Heap

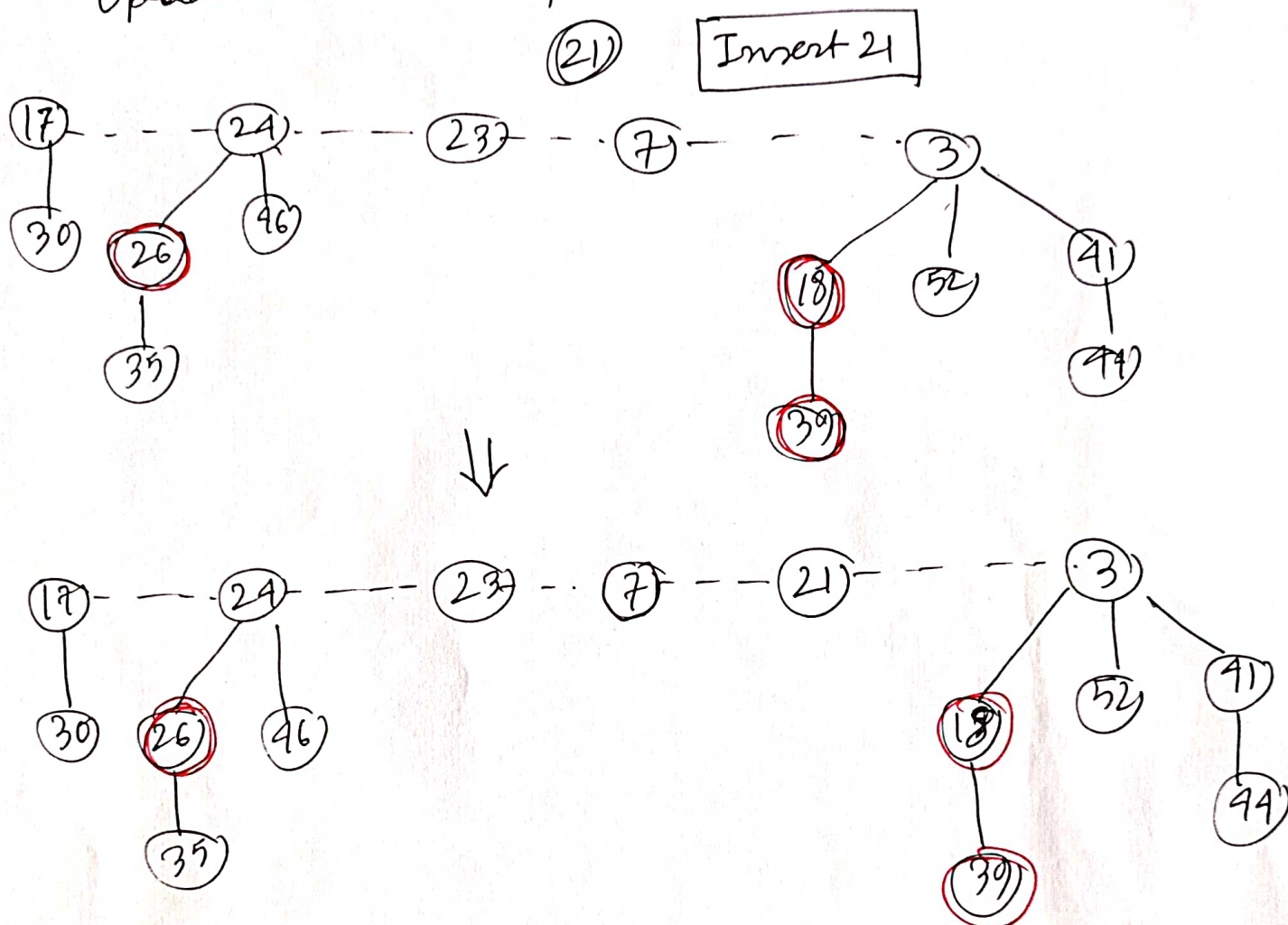
- * This create an empty Fibonacci heap
- * Create an object to store $\min[H]$ and $n[H]$
- * Initialize $\min[H] = \text{NIL}$ and $n[H] = 0$
- * Potential of the newly created heap $\Phi(H) = 0$
- * Amortized cost = actual cost = $O(1)$

Fibonacci Heap

(5)

* Inserting an Element

- Add the element to the left of $\min[H]$
- Update $\min[H]$ if needed



Amortized cost of Insert

* Actual cost $O(1)$

* Change in potential = +1

- One new ~~tree~~ node, no new marked node.

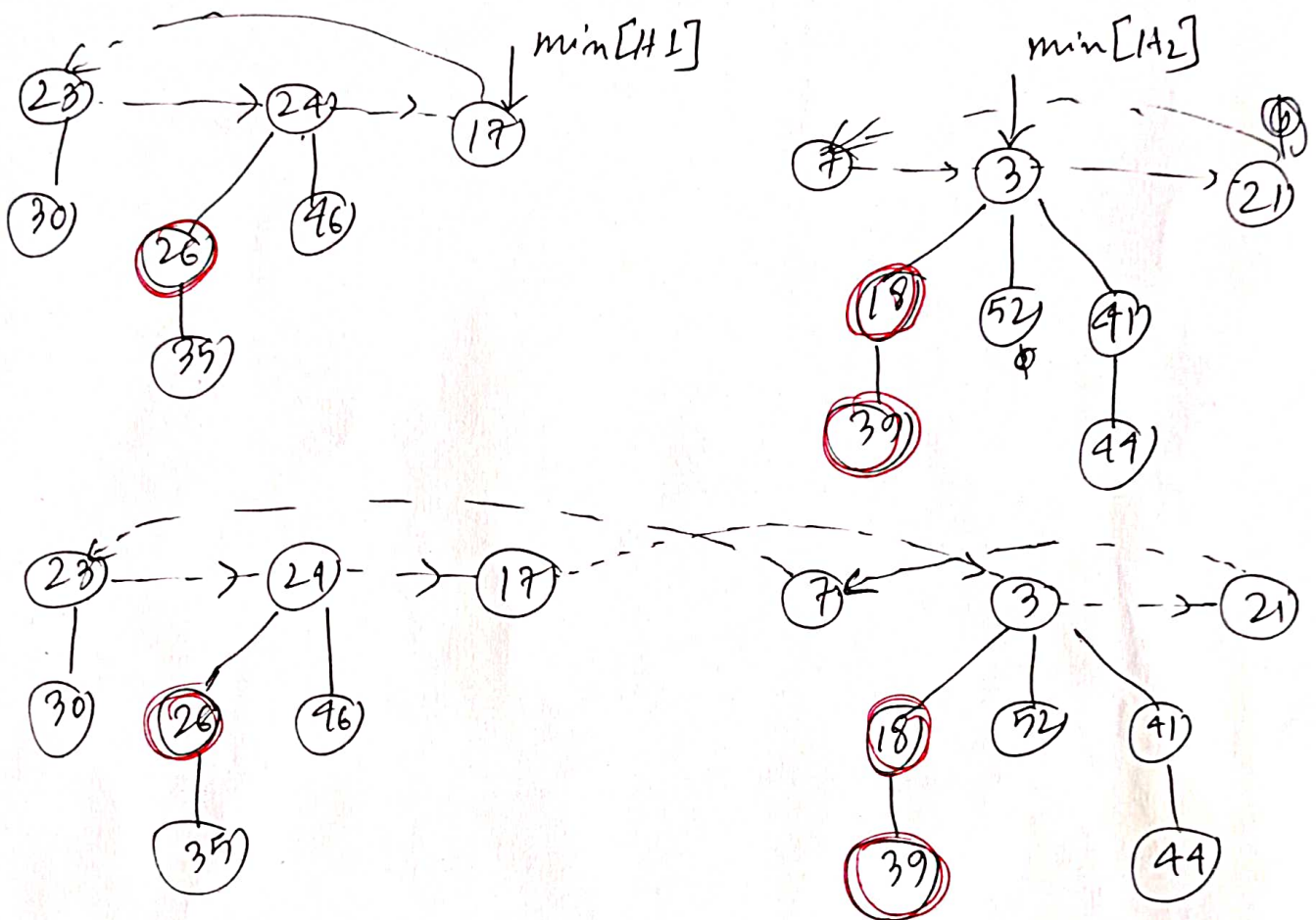
* Amortized cost = $O(1) + 1 = O(1)$

Fibonacci Heap

(6)

Merging two Heaps (Union)

- * concatenate the root lists of the two Fibonacci heaps
- * Root lists are circular, doubly linked lists, so can be easily concatenated.



Amortized Cost of Merge/Union

- * Actual Cost = $O(1)$
- * Change in potential = 0
- * Amortized cost = $O(1)$

Fibonacci Heap

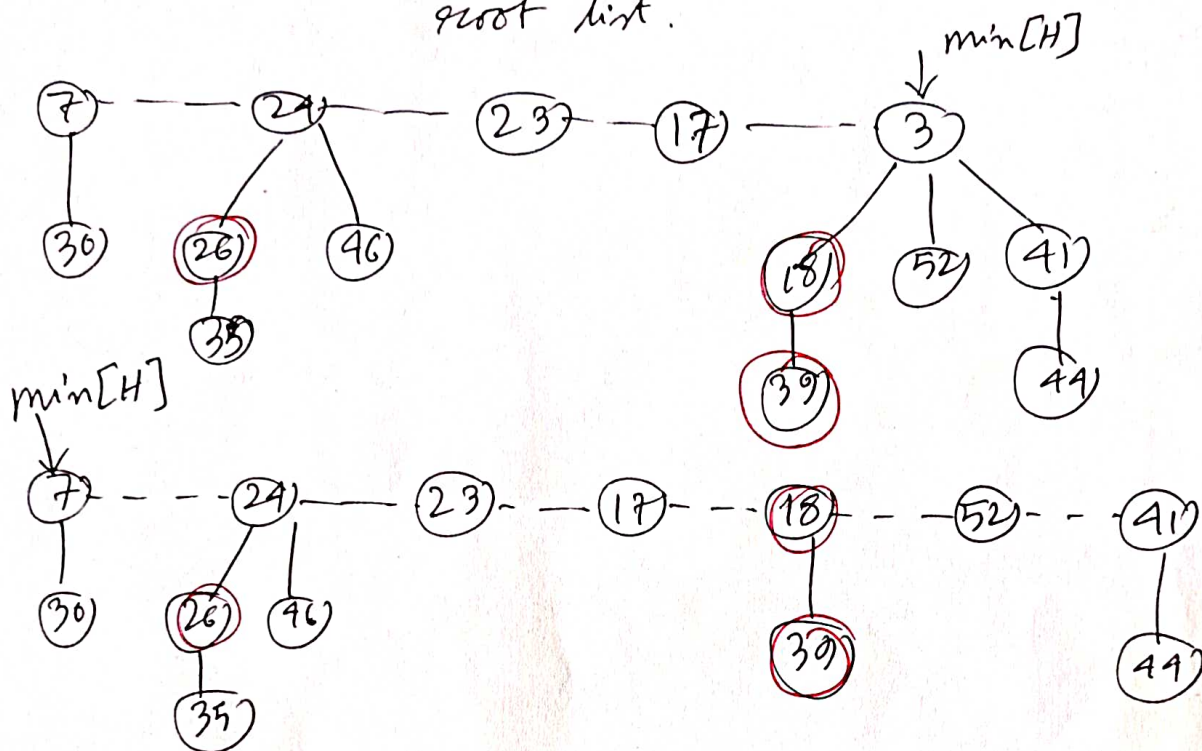
(7)

Extracting the minimum element

* Step 1:

→ Delete the node pointed by $\min[H]$

→ Concatenate the deleted node's children in to the root list.



* Step 2: Consolidate trees so that no two roots have same degree.

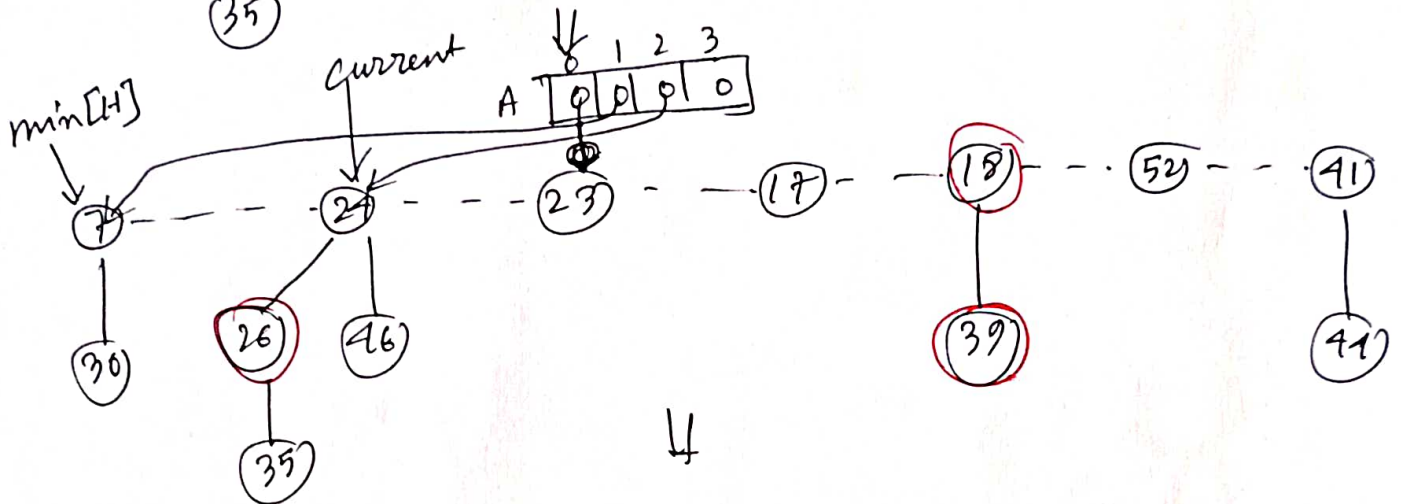
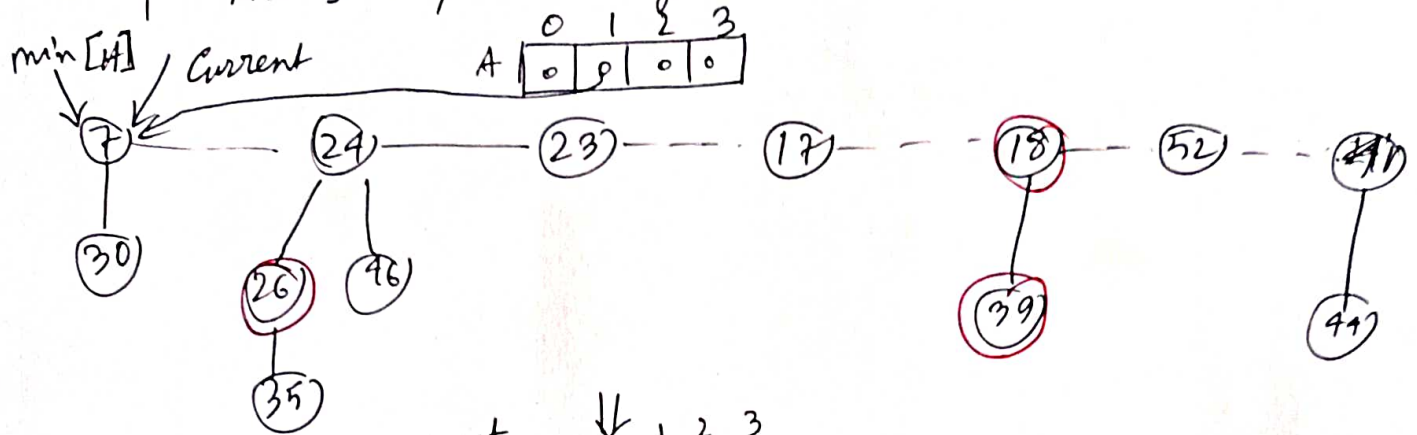
- Traverse the roots from min towards right
- Find two roots x and y with same degree, with $\text{key}[x] \leq \text{key}[y]$
- Remove y from root list and make y a child of x
- Increment $\text{degree}[x]$
- Unmarked y if marked.

* We use an array $A[0 \dots D(n)]$ where $D(n)$ is the maximum degree of any node in the heap with n nodes, initially all NIL.

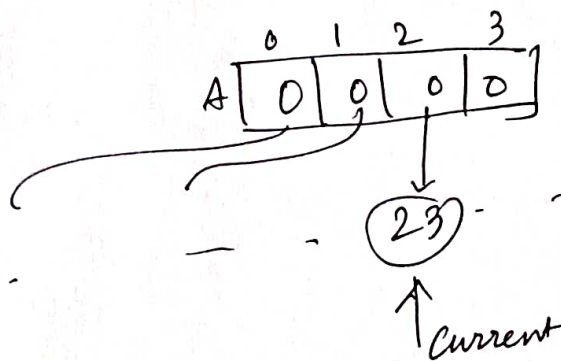
Fibonacci Heap

(8)

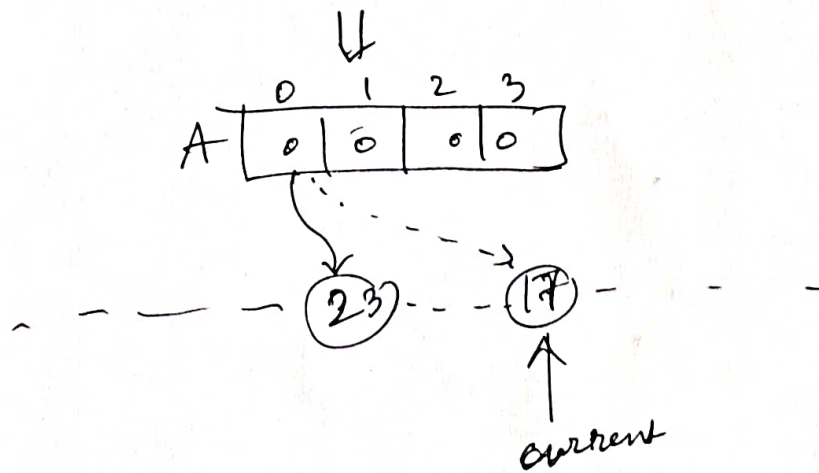
* If $A[K] = y$ at any point of time, then $\text{degree}[y] = K$



↓



↓

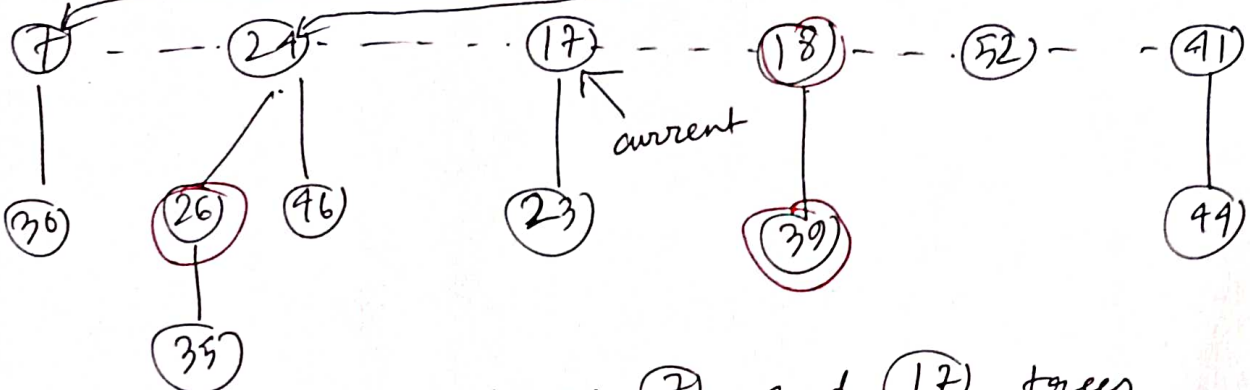


Merge (17) and (23) trees

Fibonacci Heap

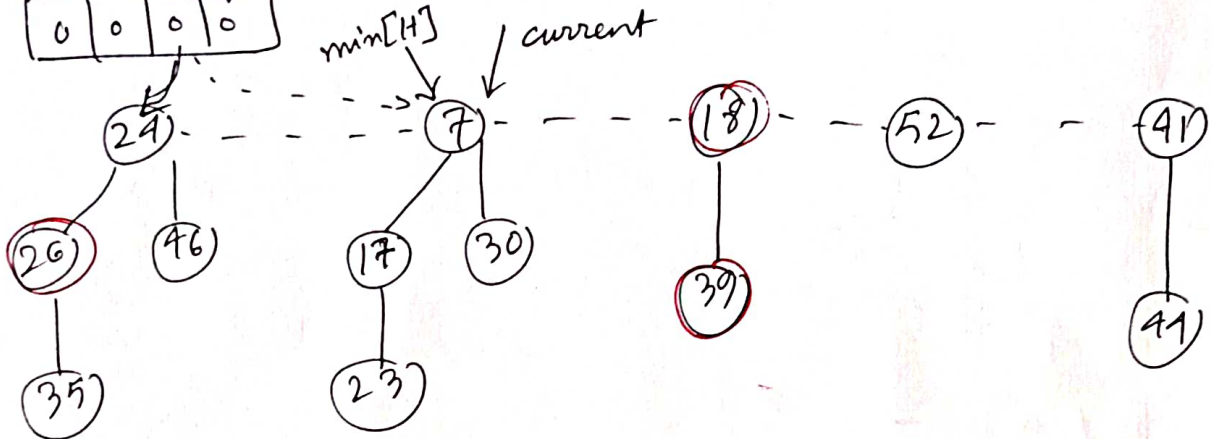
9

0	1	2	3
0	0	0	0



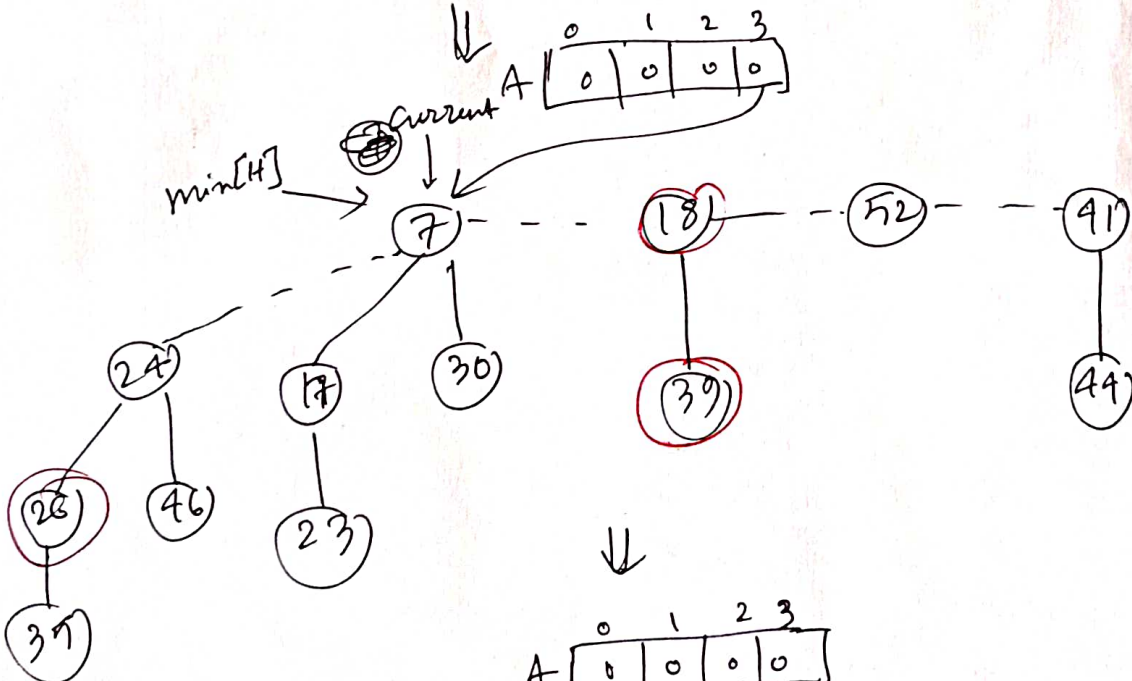
Merge (7) and (17) trees.

0	1	2	3
0	0	0	0

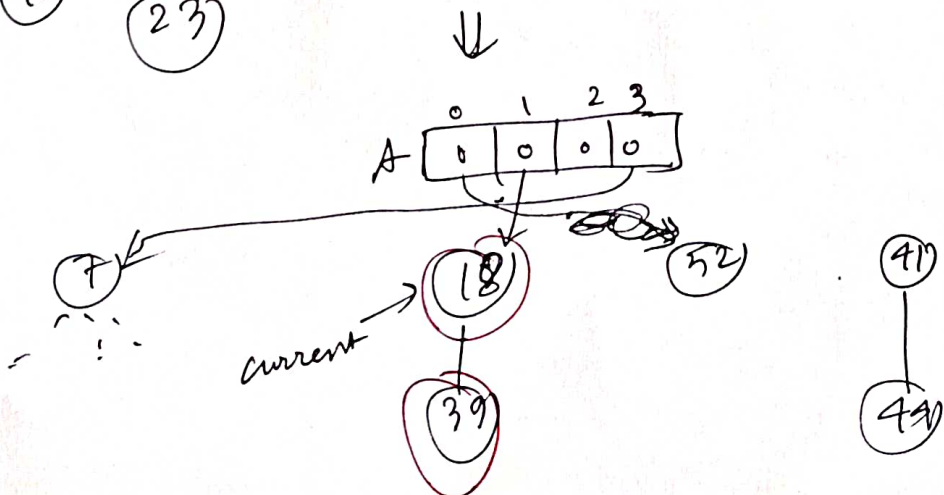


Merge (7) and (24) trees.

0	1	2	3
0	0	0	0

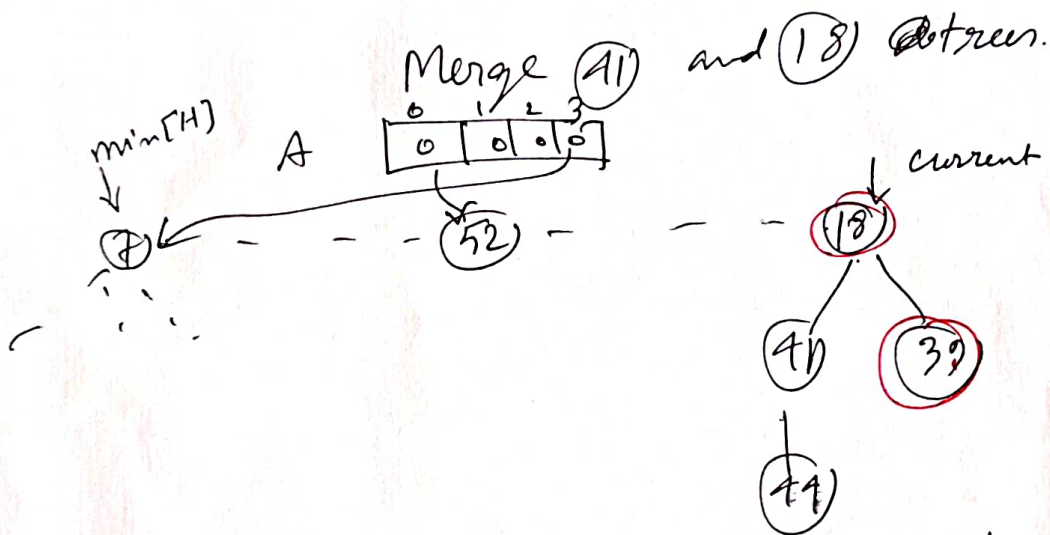
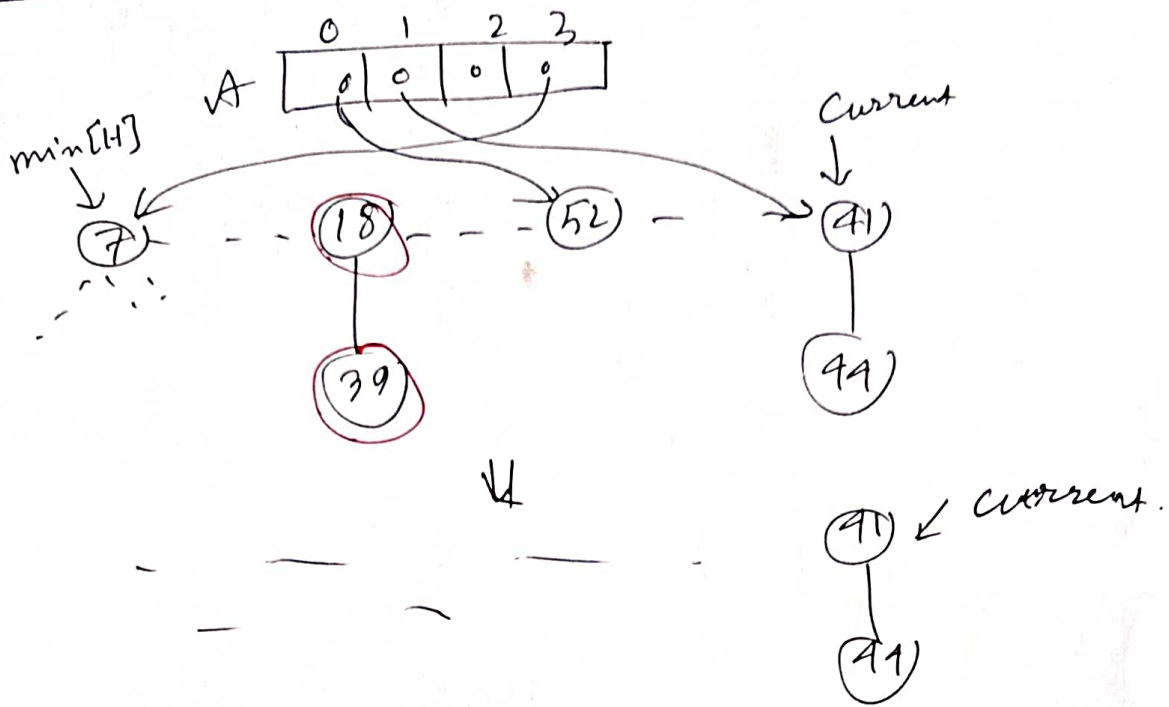


0	1	2	3
1	0	0	0

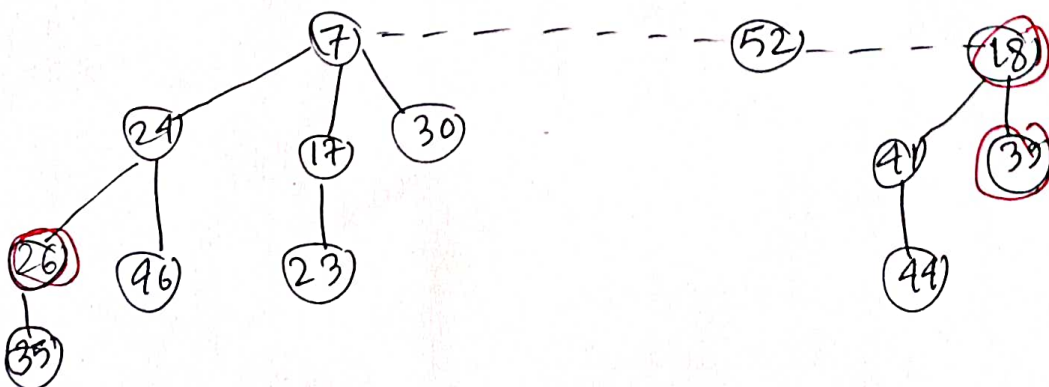


Fibonacci Heaps

(10)



- ✗ All roots are covered by current pointer, so done.
- ✗ Now find the minimum among the roots and make $\text{min}[H]$ point to that
- ✗ The Final heap is



Fibonacci Heaps

(11)

Amortized cost of Extracting Min

Recall that:

* $D(n)$ = max degree of any node in the heap with n nodes

* $t(H)$ = number of trees in heap H

* $m(H)$ = number of marked nodes in heap H

* Potential function: $\Phi(H) = t(H) + 2m(H)$

- Actual cost:

* Time for Step 1:

- $O(D(n))$ work adding min's children in to root list

* Time for Step 2 (Consolidating trees)

- Size of root list just before Step 2 is $\leq D(n) + t(H) - 1$
- $t(H)$ original roots before deletion minus the one deleted plus the number of children of the deleted nodes.

- The maximum number of merges possible is the no. of nodes in the root list

- Each merge takes $O(1)$ time.

- So total $O(D(n) + t(H))$ time for consolidation.

- $O(D(n))$ time to find the new min and updating $\text{min}[H]$ after consolidation, since at most $D(n) + 1$ nodes in the root list.
(Since only one root is possible for each degree)

* Total actual cost = time for step 1 + time for step 2

$$= O(D(n) + t(H))$$

* Potential before extracting min = $t(H) + 2m(H)$

* Potential after extracting min $\leq (D(n) + 1) + 2m(H)$

- At most $D(n) + 1$ roots are there after deletion

- Can be unmarked, but not marked.

Fibonacci Heaps

(12)

* Amortized cost = actual cost + potential change

$$= O(D(n) + t(H)) + ((D(n) + 1) + 2m(H)) - (t(H) + 2m(H))$$
$$= O(D(n))$$

* But $D(n)$ can be $O(n)$, right? That seems too costly! So is $O(D(n))$ any good?

- Can show that $D(n) = O(\log n)$ [Proof omitted.]
- So amortized cost = $O(\log n)$

Decrease Key

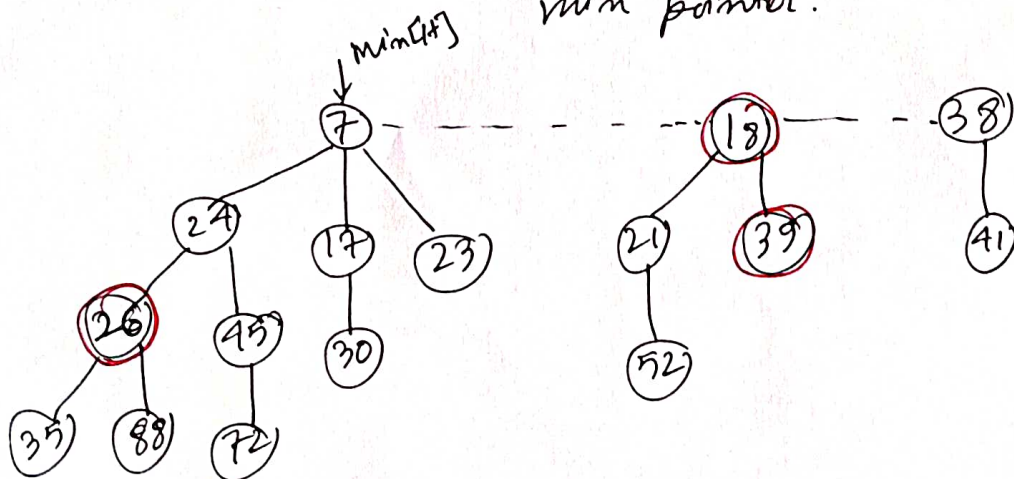
* Decrease key of element x to k

* Case 0: min heap property not ~~violated~~ violated.

- decrease key of x to k
- change heap min pointer if necessary.

* Case 1: parent of x is unmarked

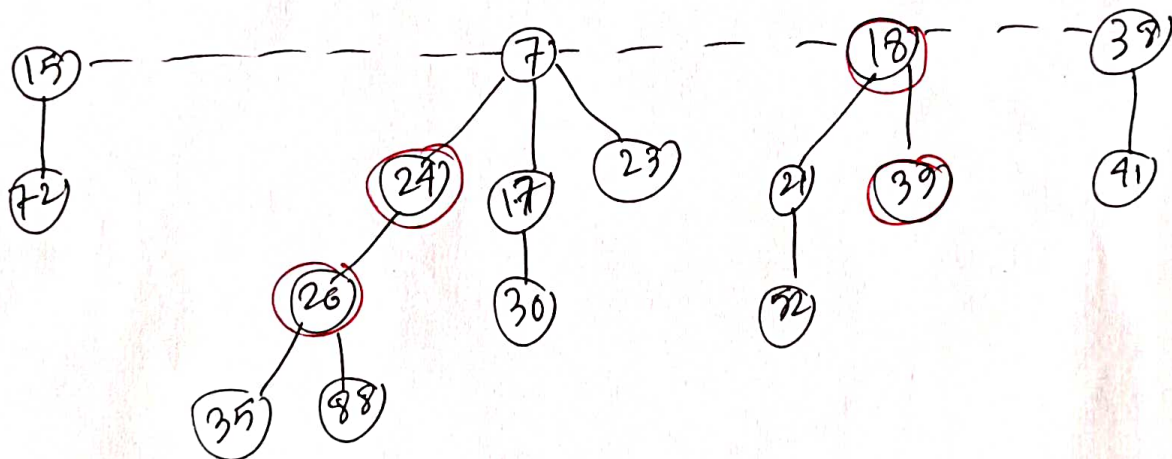
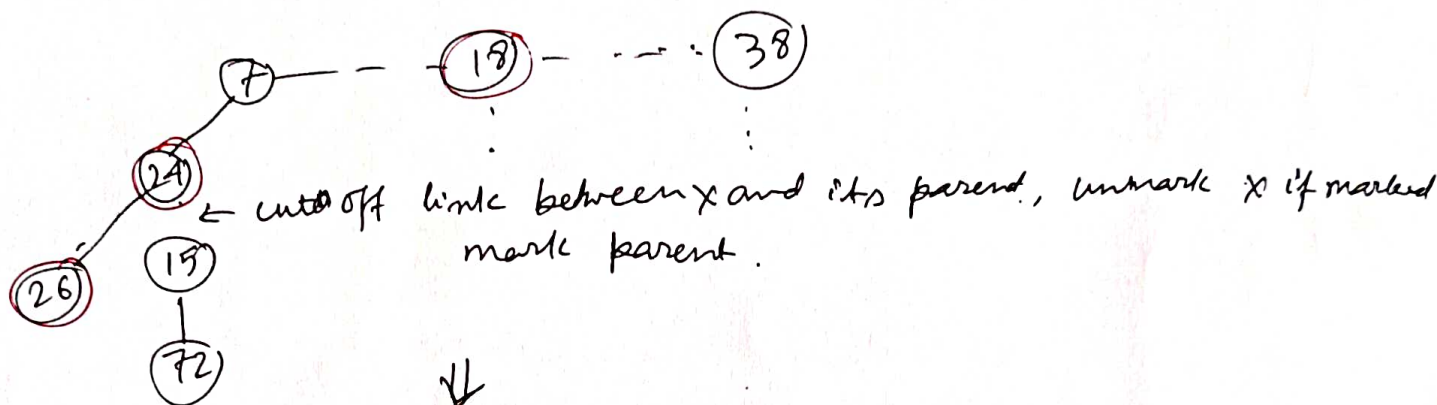
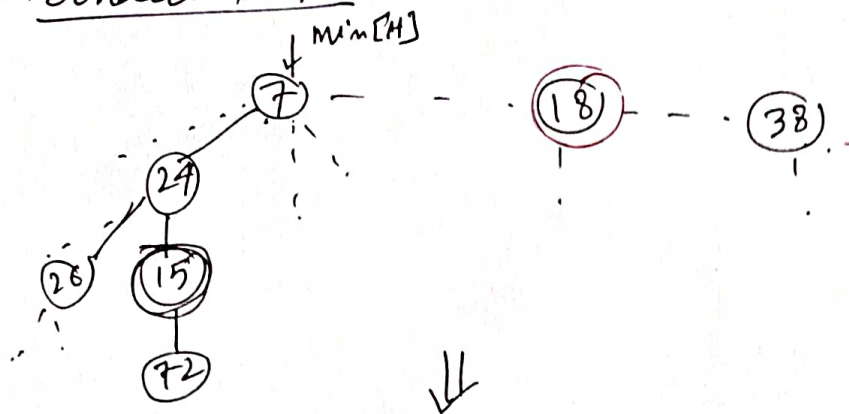
- decrease key of x to k
- cut off link between x and its parent, unmark x if marked
- mark parent
- add tree rooted at x to root list, updating min pointer.



~~Decrease~~ Decrease 45 to 15

Fibonacci Heaps

13



* Case 2: Parent of x is marked.

- decrease key of x to k
- cut off link between x and its parent $p[x]$, add x to root list, unmark x if marked.
- cut off link between $p[x]$ and $p[p[x]]$, add $p[x]$ to root list, unmark $p[x]$ ~~if marked~~.

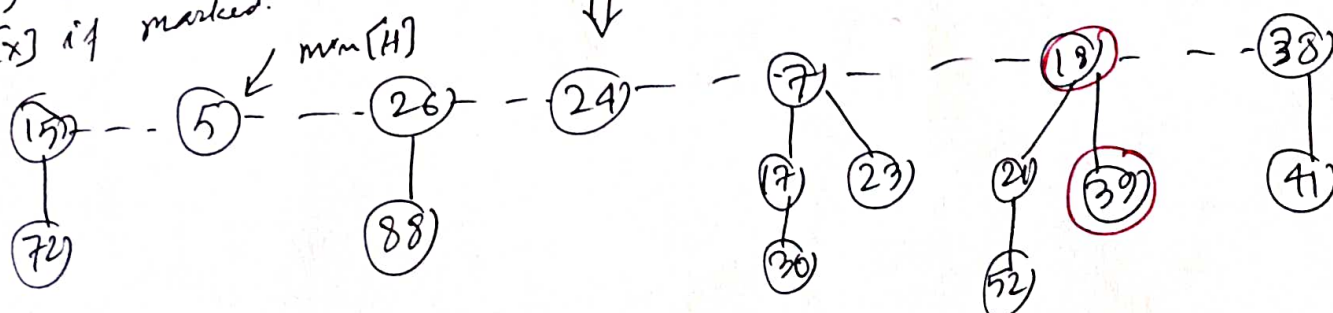
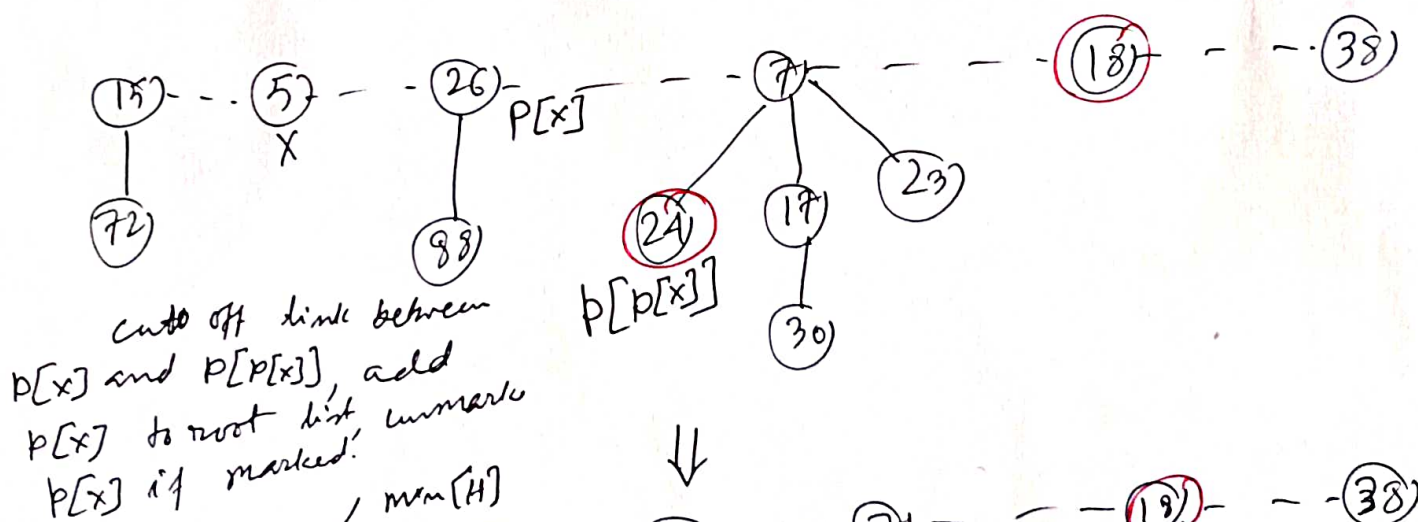
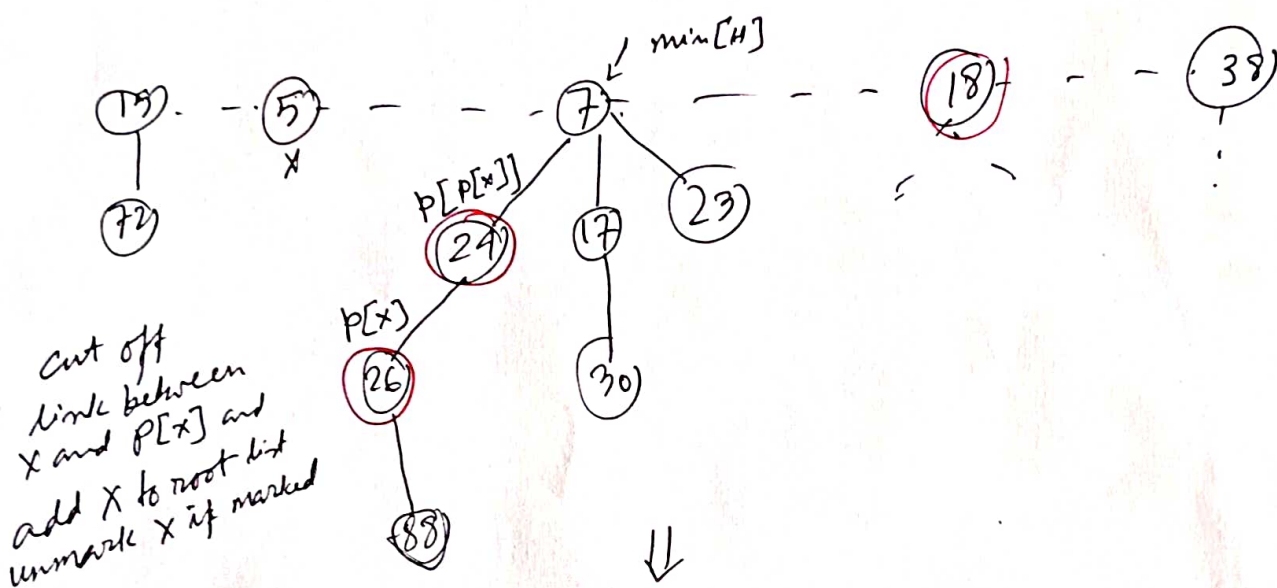
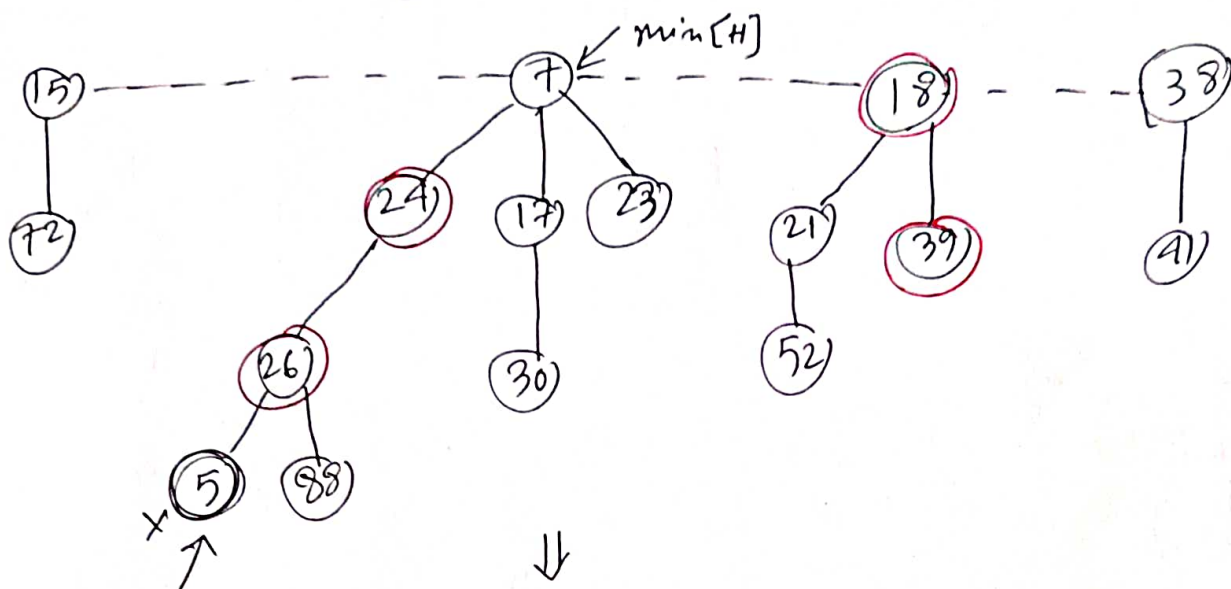
* If $p[p[x]]$ unmarked, then mark it and stop

* If $p[p[x]]$ marked, cut off $p[p[x]]$, unmark, and repeat untill unmarked node found or root reached. [cascading cut]

Fibonacci Heaps

(14)

Decrease 35 to 5



Cascading cut.

Amortized Cost of Decrease Key

* Actual cost

* $O(1)$ time for decreasing key value, and the first cut of x .

* $O(1)$ time for each of c cascading cuts, plus reinserting in root list.

* Total $O(c)$

* Change in Potential:

* H = tree just before decreasing key, H' just after.

$$* t(H') = t(H) + c$$

- ~~$t(H) + (c-1)$~~ trees from the cascading cut + the tree rooted at x .

$$* m(H') \leq m(H) - c + 2$$

- Each cascading cut unmarks a node except the last one $-(c-1)$

- Last cascading cut could potentially mark a node $(+1)$

* Change in Potential:

$$= (t(H') + 2m(H')) - (t(H) + 2m(H))$$

$$\leq c + 2(-c + 2) = 4 - c$$

* Amortized cost = actual cost + potential change.

$$= O(c) + 4 - c = O(1)$$

Fibonacci Heap

(16)

Deleting an Element :

* Delete node x .

- Decrease key of x to $-\infty$
- Delete min element in heap.

* Amortized cost

- $O(1)$ for decrease key
- $O(D(n))$ for delete-min.
- Total $O(D(n))$
 - Again can show that $D(n) = O(\log n)$
 - So, amortized cost of delete = $O(\log n)$

~~Bounded~~

Bounding the maximum degree - $D(n)$

We have to show that $D(n) \leq O(\log n)$

In particular we shall show $D(n) \leq \lfloor \log_{\phi} n \rfloor$, where ϕ is the golden ratio.

$\phi = \frac{1+\sqrt{5}}{2} \approx 1.61803$ which is positive root of the equation $x^2 = x + 1$

Fibonacci number $F_k \geq \left\lfloor \frac{\phi^k}{\sqrt{5}} + \frac{1}{2} \right\rfloor$

Key idea:- * For each node x in the heap define $\text{size}(x)$ - number of nodes including x .

* We show that $\text{size}(x)$ is exponential in ~~x~~ $\text{degree}[x]$.

Fibonacci Heap

(17)

Lemma 1: Let x be any node in a Fibonacci heap, and suppose that ~~x 's degree~~ $\text{degree}[x] = k$. Let $y_1, y_2, \dots, y_k$ denote the children of x in the order in which they were linked to x , from the earliest to the latest. Then, ~~y_1 's degree~~ $\text{degree}[y_1] \geq 0$, and $\text{degree}[y_i] \geq i-2$ for $i = 2, 3, \dots, k$.

Proof: Obviously ~~y_1 's degree~~ $\text{degree}[y_1] \geq 0$

* For $i \geq 2$, when y_i was linked to x , all of $y_1, y_2, \dots, y_{i-1}$ were children of x , so we must have ~~x 's degree~~ $\text{degree}[x] \geq i-1$.

* Because node y_i is linked to x (by consolidation) only if ~~x 's degree~~ $\text{degree}[x] = \text{degree}[y_i]$,
 $\Rightarrow$ So ~~y_i 's degree~~ $\text{degree}[y_i] \geq i-1$ at that time.

* Since then, node y_i has lost at most one child, since it would have been cut from x (by cascading cut) if it had lost two children [after one lost it became marked]

* So, ~~y_i 's degree~~ $\text{degree}[y_i] \geq i-2$

□

Remember Fibonacci Sequence.

$$F_k = \begin{cases} 0 & \text{if } k=0 \\ 1 & \text{if } k=1 \\ F_{k-1} + F_{k-2} & \text{if } k \geq 2 \end{cases}$$

Fibonacci Heap

(18)

Lemma 2: For all integer $k \geq 0$,

$$F_{k+2} = 1 + \sum_{i=0}^k F_i$$

Proof: (by induction)

$$1 + \sum_{i=0}^0 F_i = 1 + F_0 = 1 + 0 = F_2$$

Inductive Hypothesis: $F_{k+1} = 1 + \sum_{i=0}^{k-1} F_i$, so,

$$\begin{aligned} F_{k+2} &= F_k + F_{k+1} \\ &= F_k + \left(1 + \sum_{i=0}^{k-1} F_i\right) = 1 + \sum_{i=0}^k F_i \end{aligned}$$

□

Lemma 3: For all integers $k \geq 0$, the $(k+2)$ nd Fibonacci number satisfies $F_{k+2} \geq \phi^k$

Proof: (by induction)

Base cases: For $k=0$ and $k=1$, $F_2 = 1 = \phi^0$

$$F_3 = 2 > 1.618 > \phi^1$$

Hypothesis: For $k \geq 2$, assume $F_{i+2} > \phi^i$ for $i=0, 1, \dots, k-1$

$$\begin{aligned} F_{k+2} &= F_{k+1} + F_k \geq \phi^{k-1} + \phi^{k-2} \quad (\text{by inductive hypothesis}) \\ &= \phi^{k-2}(\phi + 1) \\ &= \phi^{k-2} \cdot \phi^2 \quad \left[\phi \text{ is the root of } x^2 = x + 1 \right] \\ &= \phi^k \end{aligned}$$

□

Fibonacci Heap

(19)

Lemma 4: Let x be any node in a Fibonacci heap, and let $k = \text{degree}[x]$. Then $\text{size}(x) \geq F_{k+2} \geq \phi^k$, where $\phi = (1+\sqrt{5})/2$

Proof: Let S_k denote the minimum possible size of any node of degree k in any Fibonacci heap

Trivially, $S_0 = 1$, $S_1 = 2$

The number S_k is at most $\text{size}(x)$ and, because adding children to a node can not decrease the node's size, the value of S_k increases monotonically with k .

Consider a node z , with $\text{degree}[z] = k$ and $\text{size}(z) = S_k$

Since $S_k \leq \text{size}(x)$, we compute a lower bound on $\text{size}(x)$ by computing a lower bound on S_k .

As in Lemma 1, let $y_1, y_2, \dots, y_k$ denote the children of z in the order in which they were linked to z .

To bound S_k , we count one for z itself and one for the first child y_1 (for which $\text{size}(y_1) \geq 1$) giving.

$$\begin{aligned} \text{size}(x) &\geq S_k \\ &\geq 2 + \sum_{i=2}^k S_{\text{degree}[y_i]} \\ &\geq 2 + \sum_{i=2}^k S_{i-2} \quad \left[\begin{array}{l} \text{From Lemma 1,} \\ \text{degree}[y_i] \geq i-2 \end{array} \right] \end{aligned}$$

We now show by induction on k that $S_k \geq F_{k+2}$ for all non-negative integers k .

For ~~trivial~~ base cases, $k=0$, $k=1$ it is trivial.

Fibonacci Heap

(20)

Inductive Hypothesis: We assume $k \geq 2$, and $S_i \geq F_{i+2}$ for $i = 0, 1, 2, \dots, k-1$, we have

$$\begin{aligned} S_k &\geq 2 + \sum_{i=2}^k S_{i-2} \\ &\geq 2 + \sum_{i=2}^k F_i \\ &= 1 + \sum_{i=0}^k F_i \\ &= F_{k+2} \quad (\text{by Lemma 2}) \\ &\geq \phi^k \quad (\text{by Lemma 3}) \end{aligned}$$

Thus, we have shown that $\text{size}(n) \geq S_k \geq F_{k+2} \geq \phi^k$ $\square$

Corollary 5: The maximum degree $D(n)$ of any node in an n -node Fibonacci heap is $O(\log n)$.

Proof: Let x be any node in any Fibonacci heap, and let $k = \text{degree}[x]$.

By Lemma 4, we have $n \geq \text{Size}(x) \geq \phi^k$

Taking base- ϕ log $\Rightarrow k \leq \log_{\phi} n$

Since k is an integer $\Rightarrow k \leq \lfloor \log_{\phi} n \rfloor$.

The maximum degree $D(n)$ of any node is thus $O(\log n)$.